```
Start coding or generate with AI.
```

✈️ Airline Customer Satisfaction: Decision Tree Optimization Pipeline

Hyperparameter Tuning, Interpretability Mapping, and Linear Benchmarking

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, f1_score, accuracy_score

# 1. Load data
df = pd.read_csv('4bb936ec-189a-4a96-8818-e19ac0d167e5.csv')

# 2. Clean missing data entries using median imputation to prevent data leakage
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].me

# 3. Separate target and transform into binary indicator arrays
df['target'] = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0})
df = df.drop(columns=['satisfaction'])

# 4. Apply One-Hot Encoding to categorical columns
X = pd.get_dummies(df.drop(columns=['target']), drop_first=True)
y = df['target']

print("Data cleaning and feature matrix construction complete.")
print(f"Matrix Dimension: {X.shape[0]} Rows × {X.shape[1]} Engineered Columns")
```

```
Data cleaning and feature matrix construction complete.
Matrix Dimension: 129880 Rows × 22 Engineered Columns
```

```
# Analyze target label metrics to satisfy baseline requirements
class_counts = y.value_counts()
class_props = y.value_counts(normalize=True)

print("--- Target Class Distribution Counts ---")
print(class_counts)
print("\n--- Target Class Proportional Splits ---")
print(class_props)

# Generate a cleanly sorted distribution plot
fig, ax = plt.subplots(figsize=(6, 4))
sns.barplot(x=class_counts.index, y=class_counts.values, order=class_counts.index, palette='viridis',
ax.set_title('Target Metric Distribution: Customer Satisfaction Split')
ax.set_xticklabels(['Satisfied (1)', 'Dissatisfied (0)'])
ax.set_ylabel('Passenger Observations')
plt.savefig('class_distribution.png', bbox_inches='tight')
plt.close()
```

```
--- Target Class Distribution Counts ---
target
1    71087
0    58793
```

```
Name: count, dtype: int64

--- Target Class Proportional Splits ---
target
1    0.547328
0    0.452672
Name: proportion, dtype: float64
/tmp/ipykernel_1690/2895290959.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` v

  sns.barplot(x=class_counts.index, y=class_counts.values, order=class_counts.index, palette='viridis',
/tmp/ipykernel_1690/2895290959.py:14: UserWarning: set_ticklabels() should only be used with a fixed nur
  ax.set_xticklabels(['Satisfied (1)', 'Dissatisfied (0)'])
```

```
# 80/20 train-test split with stratification to maintain class balance across matrices
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training partition shape: {X_train.shape}")
print(f"Testing partition shape : {X_test.shape}")
```

```
Training partition shape: (103904, 22)
Testing partition shape : (25976, 22)
```

```
# Design a grid space to optimize node depth and prevent leaf overfitting
param_grid = {
    'max_depth': [4, 6, 8, 10],
    'min_samples_split': [20, 50],
    'criterion': ['gini', 'entropy']
}

# Run 3-Fold Cross-Validation targeting F1 performance
grid_search = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring='f1',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)

best_tree = grid_search.best_estimator_
print("--- Optimal Decision Tree Hyperparameters ---")
print(grid_search.best_params_)
print(f"Cross-Validated Training F1-Score: {grid_search.best_score_:.4f}")
```

```
--- Optimal Decision Tree Hyperparameters ---
{'criterion': 'entropy', 'max_depth': 10, 'min_samples_split': 20}
Cross-Validated Training F1-Score: 0.9301
```

```
# Evaluate predictions on held-out test data
y_pred_tree = best_tree.predict(X_test)

tree_f1 = f1_score(y_test, y_pred_tree)
tree_acc = accuracy_score(y_test, y_pred_tree)

print("--- Decision Tree Final Test Partition Validation ---")
print(f"F1-Score for 'Satisfied' Class: {tree_f1:.4f}")
print(f"Overall Model Accuracy        : {tree_acc:.4f}\n")

# Construct Confusion Matrix Heatmap
cm_tree = confusion_matrix(y_test, y_pred_tree)
```

```
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm_tree, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted Dissatisfied', 'Predicted Satisfied'],
            yticklabels=['Actual Dissatisfied', 'Actual Satisfied'], ax=ax)
ax.set_title('Optimized Decision Tree Confusion Matrix')
plt.savefig('tree_confusion_matrix.png', bbox_inches='tight')
plt.close()
```

```
--- Decision Tree Final Test Partition Validation ---
F1-Score for 'Satisfied' Class: 0.9307
Overall Model Accuracy        : 0.9251
```

```
# Limit max_depth to 2 for clear, legible rendering and to prevent notebook parsing issues
fig, ax = plt.subplots(figsize=(16, 9))
plot_tree(best_tree, max_depth=2, feature_names=list(X.columns),
          class_names=['Dissatisfied', 'Satisfied'], filled=True, rounded=True, fontsize=10, ax=ax)
ax.set_title("Pruned Decision Tree Structural Pathways (Top 2 Tiers)", fontsize=14)
plt.savefig('decision_tree_pathways.png', bbox_inches='tight', dpi=300)
plt.close()
```

```
# Rank features by sorted Gini importance values
importances = best_tree.feature_importances_
importance_series = pd.Series(importances, index=X.columns).sort_values(ascending=False)

# Plot top 10 core service satisfaction drivers
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=importance_series.head(10).values, y=importance_series.head(10).index, palette='rocket',
ax.set_title('Top 10 Operational Metrics Driving Customer Satisfaction')
ax.set_xlabel('Relative Gini Importance Contribution Score')
plt.savefig('feature_importance_ranking.png', bbox_inches='tight')
plt.close()

print("--- Top 5 Primary Drivers identified ---")
print(importance_series.head(5))
```

```
/tmp/ipykernel_1690/3964986065.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` v

  sns.barplot(x=importance_series.head(10).values, y=importance_series.head(10).index, palette='rocket'
--- Top 5 Primary Drivers identified ---
Inflight entertainment              0.385860
Seat comfort                        0.233716
Ease of Online booking              0.072102
Departure/Arrival time convenient   0.047342
On-board service                    0.046291
dtype: float64
```

```
# Scaling inputs for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train a baseline Logistic Regression model
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
y_pred_lr = lr_model.predict(X_test_scaled)

lr_f1 = f1_score(y_test, y_pred_lr)
lr_acc = accuracy_score(y_test, y_pred_lr)

print("--- Comparative Model Metric Portfolio ---")
```

```
print(f"Decision Tree Accuracy : {tree_acc*100:.2f}% | F1-Score: {tree_f1:.4f}")
print(f"Logistic Regression Acc: {lr_acc*100:.2f}% | F1-Score: {lr_f1:.4f}")
```

```
--- Comparative Model Metric Portfolio ---
Decision Tree Accuracy : 92.51% | F1-Score: 0.9307
Logistic Regression Acc: 82.89% | F1-Score: 0.8436
```

Start coding or generate with AI.